

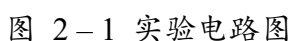
一、实验任务及目的

- 熟悉微程序控制电路、执行流程和指令系统;
- 跟踪各种操作模式的执行过程;

- 掌握微程序控制器的原理；
- 掌握 TEC-Plus / TEC-8 模型计算机中微程序控制器的实现方法和微地址转移逻辑的实现方法；

首先需要了解整体实验电路, 实验电路图如图 2-1 所示;

本次实验用到了 TEC-8 模型计算机的微程序控制电路模块；需要将开关切换至“微程序”行实验。



TEC-8 计算机中，指令格式长度固定为 8 位，对应 8 位指令寄存器 IR；在程序执行时：

1. 程序开始执行，从程序计数器 PC 中取指令并存入指令寄存器 IR；
2. 指令寄存器 IR 中，高 4 位 IR7 ~ IR4 被送入微程序控制器，用于产生相应的控制信号；也是本次实验所用到的；
3. 低 4 位 IR3 ~ IR0 和 SEL3 ~ SEL0 通过 2 选 1 选择器的 SELECT 信号进行选择，输出 4 位 RD1、RD0、RS1、RS0，即为前面实验中使用到的寄存器的片选信号；

其次介绍微程序控制电路的基本内容：

(2) 微指令格式：

在 TEC-8 模型计算机中，采用长度为 40 位（包含控制字段 29 位和顺序字段 11 位）的微指令格式如图 2-2 所示；

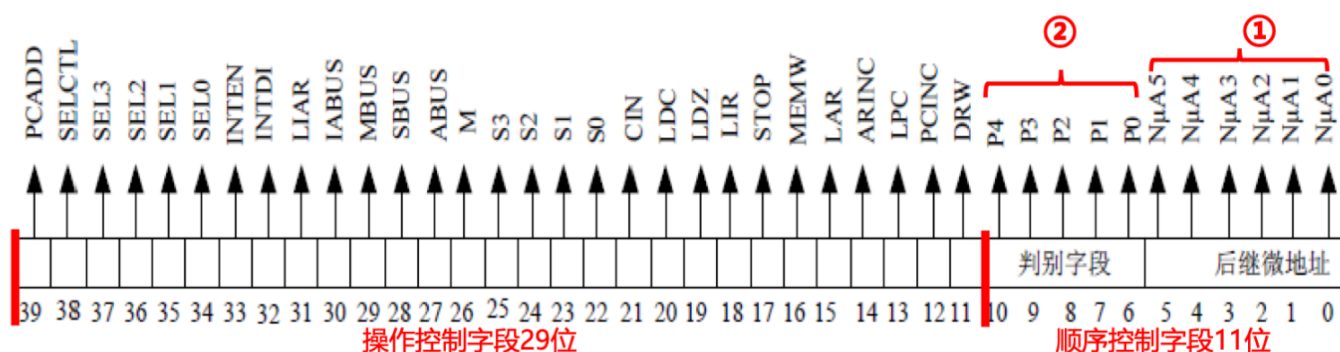


图 2-2 微指令格式

(3) 微程序流程分析：

见第三部分。

(4) 微程序控制电路分析：

1. 最上面的 CM0 ~ CM4 组成控制存储器部分，由 5 片 58C65 (8K * 8 位的 E²PROM)，用于存储微指令，对应 40 位微指令，其中 CM0 用于存储低 8 位，CM4 用于存储高 8 位；正常工作状态下处于只读状态；

2. 中间部分的 REG6D + D 触发器组成微地址寄存器。该部分保存当前执行的微地址 (μA0 ~ μA5)。在一条微指令结束时，用 T3 的下降沿控制更新，将微地址转移逻辑产生的下一条微指令地址 NμA0 - T ~ NμA5 - T 写入微地址寄存器；

3. 最下面由多个与、或门电路组成微地址转移逻辑电路，负责决定下一条微指令的地址。其输入信号包括当前微指令的下址和系统控制信号等，形成新的微地址编码 (NJ μA0 ~ NJ μA5)；

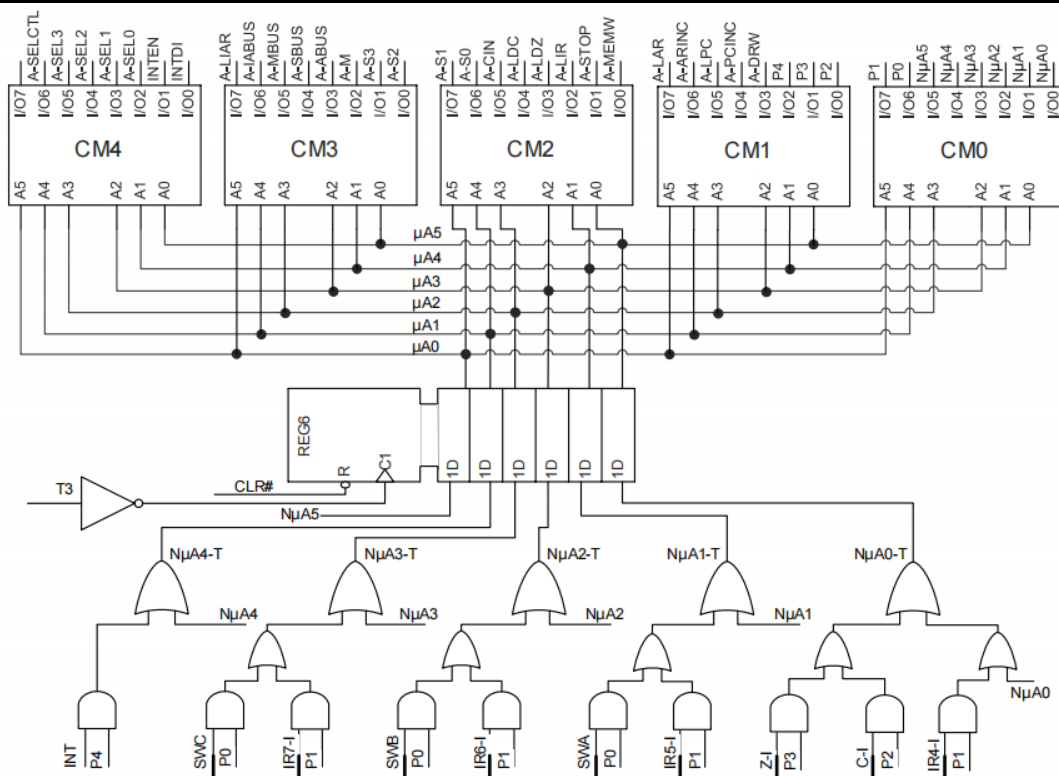


图 2-3 微程序控制电路图

注：T3 是一个节拍脉冲中的最后一个脉冲，使用 T3 的下降沿，保证了当前微指令执行完毕后再更新微地址，防止控制信号出错；

接下来，我们简要介绍实验中涉及的判别字段、操作模式等及其含义：

(5) 微指令字段含义解释：

相当于确定微指令的寻址方式。如表 2-1 所示。

字段	解释
NμA0 ~ NμA5	下址，在微指令顺序执行的情况下，它是下一条微指令的地址；
P0	= 1 时，根据后继微地址 NμA5 ~ NμA0 和模式开关 SWC、SWB、SWA 确定下一条微指令的地址；
P1	= 1 时，根据后继微地址 NμA5 ~ NμA0 和指令操作码 IR7 ~ IR4 确定下一条微指令的地址；
P2	= 1 时，根据后继微地址 NμA5 ~ NμA0 和进位 C 确定下一条微指令的地址；
P3	= 1 时，根据后继微地址 NμA5 ~ NμA0 和零标志 Z 确定下一条微指令的地址；
P4	= 1 时，根据后继微地址 NμA5 ~ NμA0 和中断信号 INT 确定下一条微指令的地址。模型计算机中，中断信号 INT 由时序发生器在接到中断请求信号后产生；

表 2-1 微指令字段含义解释

(6) 操作模式选择:

实验中需要通过操作模式开关 SWC、SWB、SWA 来确定操作模式，具体说明如表 2-2 所示。

操作模式 (SWC,SWB,SWA)	实验功能
000	启动程序运行
001	写存储器
010	读存储器
011	读寄存器
100	写寄存器

表 2-2 操作模式说明

三、微程序流程图分析

微程序的执行流程如下，如图 2-2 所示：

1. 每次复位后，从微地址 00 开始进入执行流程；
2. 对 P0 进行条件判断，依据 SEL3~SEL0 的取值判断系统状态；
3. 根据 SWC、SWB、SWA 的不同组合（如 000, 011 等），系统将跳转至不同的操作模式；
4. 接下来，按照图中的箭头指向，系统逐条顺序执行微指令，依靠内部总线（SBUS、MBUS）、控制信号（如 LAR, LIR, ARINC, MEMW）等完成各类微操作；
5. 如果操作模式为 000（即取指模式），系统需要从存储器中取出指令，并根据指令寄存器 IR 的高 4 位（IR7~IR4）进行译码，对 P1 进行条件判断，选择对应的微程序，并跳转到对应的微地址段执行相应指令的微操作。
6. 执行完对应的微程序后，系统会对 P4 进行条件判断（如判断是否满足跳转、中断、结束等），再根据结果继续执行下一条指令，形成完整的指令周期。

以执行写寄存器操作为例进行分析：

1. 按下 CLR，从微地址 00 开始进入执行流程；
2. 对 P0 进行条件判断，此时 SWC-A=100，故执行微地址 09 的微指令；
3. 发出对应的控制信号，结合实验电路图分析即可发现此时数据开关打开，并且选中 R0 寄

寄存器，同时发出写信号 DRW；所以该微指令的功能是将数据开关的数写入寄存器 R0；

- 4. 注意执行完毕后有一个 STOP 指令，用于中断控制；
- 5. 后面微地址 08 ~ 0C 对应的微指令类似，分别将数据开关的数写入寄存器 R1 ~ R3；
- 6. 执行完毕后，回到微地址 00 处；

所以，写寄存器的操作事实上是按顺序将 R0 ~ R3 依次写入。

同样可以分析其他操作模式：

- 读寄存器操作依次执行 R0、R1 读出至运算器的 A、B 端口，R2、R3 读出至运算器的 A、B 端口；
- 读存储器操作实现地址的连续读出（注意循环）；
- 写存储器操作实现地址的连续写入（注意循环）；

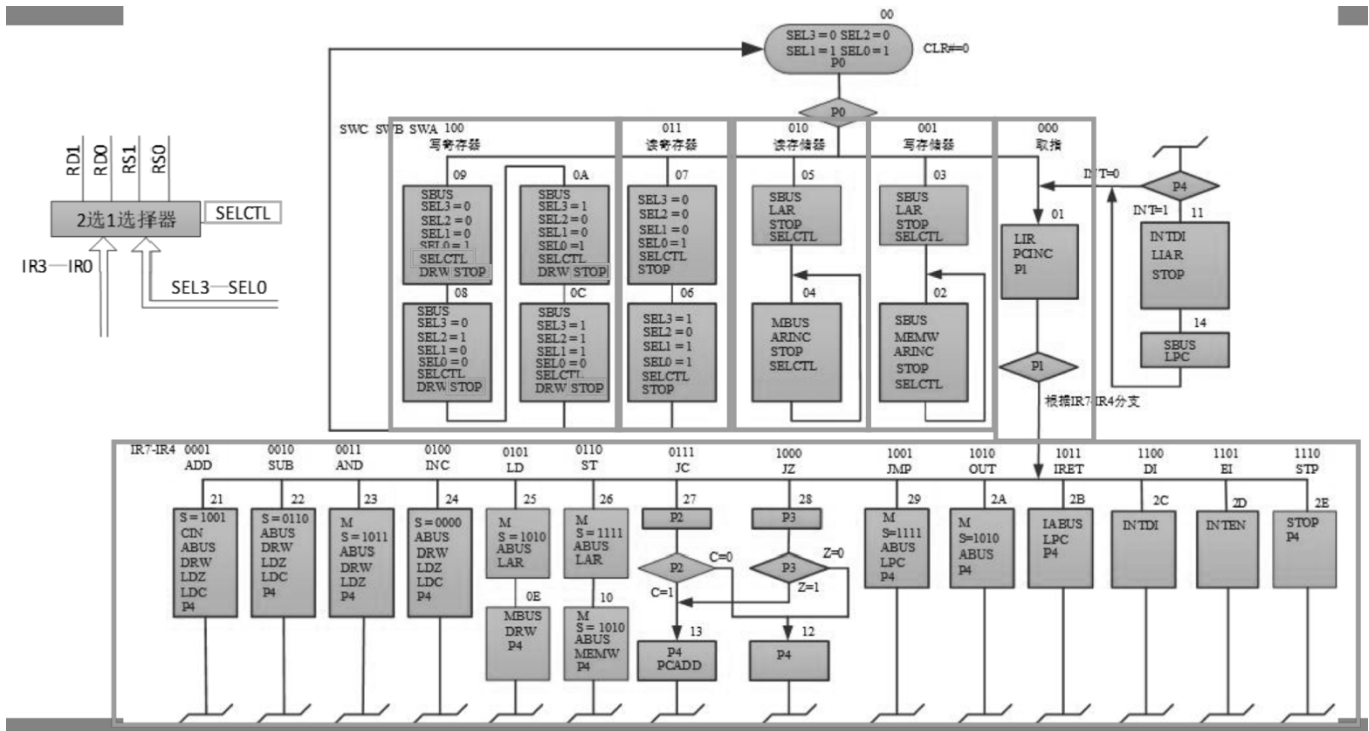


图 3-1 微程序流程图

四、 实验过程及结果

实验过程：

1. 实验准备：断电后，切换控制器转换开关至“微程序”，编程开关至“0”，DP=1，电路连线如表 4-1 所示；
2. 打开电源，按下复位按钮 CLR；

控制器	IR4-I	IR5-I	IR6-I	IR7-I	C-I	Z-I	CS0
电平开关	K0	K1	K2	K3	K4	K5	GND
接线颜色	蓝	蓝	蓝	蓝	黄	黄	绿

表 4-1 实际接线情况

3. 拨动操作模式开关 SWC、SWB、SWA 到目标位置（操作模式说明见表 2-2），按 QD，进入目标控制台操作模式；依次跟踪四种控制台操作模式；关于跟踪过程的具体操作见表 4-2；

4. 按复位按钮 CLR ；

5. 设置 SWC=0, SWB=0, SWA=0，按 QD，进入启动程序运行模式；

6. 依次跟踪指令执行；结束一条指令后，按 CLR，结束本次跟踪，开始下一条指令；关于指令的跟踪过程的具体操作见表 4-3；

7. 断电，整理实验仪器，实验结束；

操作模式SWC、SWB、SWA	$\mu A(\mu A5 - \mu A0)$	$N\mu A(N\mu A5 - N\mu A0)$	P(P4 - P0)	备注
CLR	000000 $\Leftrightarrow$ 00	000001 $\Leftrightarrow$ 01	00001,P0亮	程序复位
100 (QD)	001001 $\Leftrightarrow$ 09	001000 $\Leftrightarrow$ 08	00000	将数27H写入寄存器R0
100 (QD)	001000 $\Leftrightarrow$ 08	001010 $\Leftrightarrow$ 0A	00000	将数09H写入寄存器R1
100 (QD)	001010 $\Leftrightarrow$ 0A	001100 $\Leftrightarrow$ 0C	00000	将数20H写入寄存器R2
100 (QD)	001100 $\Leftrightarrow$ 0C	000000 $\Leftrightarrow$ 00	00000	将数07H写入寄存器R3
011 (QD)	000111 $\Leftrightarrow$ 07	000110 $\Leftrightarrow$ 06	00000	从寄存器R0、R1中读出数27H、09H，并通过ALU的A端口和B端口验证
011 (QD)	000110 $\Leftrightarrow$ 06	000000 $\Leftrightarrow$ 00	00000	从寄存器R2、R3中读出数20H、07H，并通过ALU的A端口和B端口验证
001 (QD)	000011 $\Leftrightarrow$ 03	000010 $\Leftrightarrow$ 02	00000	写存储器的[20H]地址，并通过AR验证
001 (QD)	000010 $\Leftrightarrow$ 02	000010 $\Leftrightarrow$ 02	00000	将数27H写入存储器的[20H]地址
001 (QD)	000010 $\Leftrightarrow$ 02	000010 $\Leftrightarrow$ 02	00000	将数09H写入存储器的[21H]地址
CLR	000000 $\Leftrightarrow$ 00	000001 $\Leftrightarrow$ 01	00001,P0亮	程序复位
010 (QD)	000101 $\Leftrightarrow$ 05	000100 $\Leftrightarrow$ 04	00000	从存储器的[20H]地址读出数据27H到数据总线上，并通过DBUS验证
010 (QD)	000101 $\Leftrightarrow$ 05	000100 $\Leftrightarrow$ 04	00000	从存储器的[21H]地址读出数据09H到数据总线上，并通过DBUS验证
CLR	000000 $\Leftrightarrow$ 00	000001 $\Leftrightarrow$ 01	00001,P0亮	程序复位

表 4-2 操作模式跟踪过程记录

指令IR7-IR4	$\mu A(\mu A5 - \mu A0)$	$N\mu A(N\mu A5 - N\mu A0)$	P(P4 - P0)	备注
QD	000001 $\Leftrightarrow$ 01	100000 $\Leftrightarrow$ 20	00010,P1亮	进入取指操作
0001 (QD)	100001 $\Leftrightarrow$ 21	000001 $\Leftrightarrow$ 01	10000,P4亮	ADD指令, 执行完后P4亮表示进入中断
0010 (QD)	100010 $\Leftrightarrow$ 22	000001 $\Leftrightarrow$ 01	10000,P4亮	
0011 (QD)	100011 $\Leftrightarrow$ 23	000001 $\Leftrightarrow$ 01	10000,P4亮	SUB指令
0100 (QD)	100100 $\Leftrightarrow$ 24	000001 $\Leftrightarrow$ 01	10000,P4亮	AND指令
0101 (QD)	100101 $\Leftrightarrow$ 25	001110 $\Leftrightarrow$ 0E	00000	LD指令
0101 (QD)	001110 $\Leftrightarrow$ 0E	000001 $\Leftrightarrow$ 01	10000,P4亮	
0110 (QD)	100110 $\Leftrightarrow$ 26	010000 $\Leftrightarrow$ 10	00000	ST指令
0110 (QD)	010000 $\Leftrightarrow$ 10	000001 $\Leftrightarrow$ 01	10000,P4亮	
0111 (QD) , C=0	100111 $\Leftrightarrow$ 27	010010 $\Leftrightarrow$ 12	00100,P2亮	JC指令 C=0
0111 (QD) , C=0	010010 $\Leftrightarrow$ 12	000001 $\Leftrightarrow$ 01	10000,P4亮	
0111 (QD) , C=1	100111 $\Leftrightarrow$ 27	010010 $\Leftrightarrow$ 12	00100,P2亮	JC指令 C=1
0111 (QD) , C=1	010011 $\Leftrightarrow$ 13	000001 $\Leftrightarrow$ 01	10000,P4亮	
1000 (QD) , Z = 0	101000 $\Leftrightarrow$ 28	010010 $\Leftrightarrow$ 12	01000,P3亮	JZ指令 Z=0
1000 (QD) , Z = 0	010010 $\Leftrightarrow$ 12	000001 $\Leftrightarrow$ 01	10000,P4亮	
1000 (QD) , Z = 1	101000 $\Leftrightarrow$ 28	010010 $\Leftrightarrow$ 12	01000,P3亮	JZ指令 Z=1
1000 (QD) , Z = 1	010011 $\Leftrightarrow$ 13	000001 $\Leftrightarrow$ 01	10000,P4亮	

表 4-3 指令跟踪过程记录

注：在跟踪不同指令切换时省略了 CLR 复位操作的记录。

实验结果分析及思考：

容易验证，本次实验对操作模式和不同指令的跟踪，每一条微指令的微地址、判别位、下地址以及对应的控制信号亮灯情况均与 TEC-8 模型计算机的微指令流程图一致。完成了对于实验任务的验证。

值得注意的是，在程序顺序执行时，下地址就是下一条微指令的微地址。但是如果需要进行条件转移判断，比如实验中 P0 = 1 时选择操作模式，以及 C-I 和 Z-I 选择 JC 和 JZ 指令模式中，下地址（即 N μ A 字段）并不直接是下一条微指令的地址，而需要通过微地址转移逻辑电路（见第二部分）计算得出。

五、 实验收获及体会

微程序控制器相较于硬布线控制器，具有规整性、易于设计和时序发生器简单等优势，被广泛应用。通过本次实验，我很好地掌握了微程序控制器的基本原理以及 TEC-8 模型计算机中微程序控制器的具体操作，很好地理解并熟悉了微程序的顺序执行以及条件转移的过程。

通过理论课的学习，还可以发现，本次实验中 TEC-8 指令系统的指令格式还可以进一步压缩长度，比如 PCADD、LPC 和 PCINC 等互斥的指令可以压缩编码，节省空间。

实验五、CPU 组成与机器指令的执行

一、 实验任务及目的

(1) 实验任务：

- 进一步熟悉 TEC-8 模型计算机的指令系统，对程序进行手动汇编（预习任务）；
- 通过简单的连线，用微程序控制器控制数据通路，构成能够运行程序的 TEC-8/TEC-Plus 模型计算机；
- 熟悉“存储程序式”思想，将程序写入存储器，给寄存器 R2、R3 赋初值；
- 进一步熟悉跟踪程序执行状态的操作：跟踪执行程序，用单拍方式运行一遍，用连续方式运行一遍，详细记录实验过程及结果；
- 用实验台操作检查并验证程序运行结果；

(2) 实验目的：

- 用微程序控制器控制数据通路，将相应的信号线连接，构成一台能够运行测试程序的 CPU ；
- 执行一个简单的程序，掌握机器指令与微指令的关系；
- 理解计算机如何取出指令、如何执行指令、如何在一条指令执行结束之后自动取出下一条指令并执行，从而牢固建立计算机整机概念；

二、 程序的手动汇编结果

通过参考 Tec-Plus/Tec-8 模型计算机的指令系统，对给定的程序进行手动汇编，程序的汇编结果如表 2-1 所示。

地址	指令	二进制机器代码	备注
00H	LD R0,[R3]	0101 0011	
01H	INC R3	0100 1100	
02H	LD R1,[R3]	0101 0111	
03H	SUB R0,R1	0010 0001	
04H	JZ 0BH	1000 0110	PC = 04H + 1 = 05H offset = 0BH - 05H = 06H = 0110
05H	ST R0,[R2]	0110 1000	
06H	INC R3	0100 1100	
07H	LD R0,[R3]	0101 0011	
08H	ADD R0,R1	0001 0001	
09H	JC 0CH	0111 0010	PC = 09H + 1 = 0AH offset = 0CH - 0AH = 02H = 0010
0AH	INC R2	0100 1000	

0BH	ST R2,[R2]	0110 1010	
0CH	AND R0,R1	0011 0001	
0DH	OUT R2	1010 0010	
0EH	STP	1110 0000	
0FH	85H	1000 0101	
10H	23H	0010 0011	
11H	EFH	1110 1111	
12H	00H	0000 0000	
13H	00H	0000 0000	

表 2-1 程序的手动汇编结果

三、实验过程及结果

本实验与之前的四个实验有较大区别，要求将时序发生器、通用寄存器组、存储器、算术逻辑运算部件以及微程序控制器组合在一起，构成一台能运行程序的简单计算机。

本实验中，数据通路通过微程序控制器进行控制，通过微程序解释程序指令的执行过程。由微程序完成从取指到执行的整个过程。

程序装入存储器以及给寄存器赋初值的操作需要在实验开始时手工完成。

实验结果需要通过控制台的有关指示灯和信号进行检查。

实验过程：

1. 实验准备：断电后，切换控制器转换开关至“微程序”，编程开关至“0”，DP=1（先进入单拍模式跟踪程序执行），电路连线如表 4-1 所示：

数据通路	IR4-I	IR5-I	IR6-I	IR7-I	C-I	Z-I	CS0
开关	IR4-O	IR5-O	IR6-O	IR7-O	C-O	Z-O	GND
接线颜色	蓝	蓝	蓝	蓝	蓝	蓝	红

表 4-1 实际接线情况

注：本次实验将指令的输入和输出、标志位的输入和输出直接相连，实现了数据通路通过微程序控制器进行控制；

2. 打开电源，进入单拍模式；

【单拍模式】

3. 通过写存储器操作将手工汇编的程序写入存储器：

3.1 按下 CLR 复位按钮；

3.2 将操作模式开关 SWC ~ SWA 设置为 001，表示“写存储器”模式；

3.3 按下 QD，将数据开关设置为 00H，再一次按下 QD，将 00H 写入地址寄存器 AR；

3.4 按照表 2-1，依次将数据开关设置为每条指令的二进制码，并按下 QD 写入存储器的对应地址单元；

4. 通过读存储器操作，将指令逐条读出，检查写入是否正确：

4.1 按下 CLR 复位按钮；

4.2 将操作模式开关 SWC ~ SWA 设置为 010，表示“读存储器”模式；

4.3 将数据开关设置为 00H，按下 QD，将 00H 写入地址寄存器 AR；

4.4 依次读出指令，验证写入程序的正确性；

注：实际实验过程中，通过 D7 ~ D0 验证每次按下 QD 后读出指令的正确性；

5. 通过写寄存器操作，将数 12H 写入寄存器 R2，0FH 写入寄存器 R3：

5.1 按下 CLR 复位按钮；

5.2 将操作模式开关 SWC ~ SWA 设置为 100，表示“写寄存器”模式；

5.3 按下 QD，将数据开关设置为任意值，按下 QD，重复 2 次（写寄存器是按顺序依次写入，所以跳过 R0、R1）；

注：实验中为避免次数数错，通过观察 SEL3 ~ SEL2 指示灯判断此时正在写入哪个寄存器；实验过程中少按了一次，但是通过步骤 6 检查出；

5.4 将数据开关设置分别设置为 12H 和 0FH 并按下 QD，将数据写入 R2 和 R3；

6. 通过读寄存器，检查写入寄存器的数据是否正确：

6.1 按下 CLR 复位按钮；

6.2 将操作模式开关 SWC ~ SWA 设置为 011，表示“读寄存器”模式；

6.3 按下 2 次 QD，读出 R2 和 R3 的值；

注：R2 和 R3 的值可以分别通过 A 端口和 B 端口的指示灯读出；

7. 启动程序，在单微指令下运行程序并记录过程数据：

7.1 按下 CLR 复位按钮；

7.2 将操作模式开关 SWC ~ SWA 设置为 000，表示“启动程序运行”模式；

7.3 按下 QD，跟踪程序执行，并依次记录中间状态信息，见表 4-2 单拍跟踪过程记录表；

8. 读取四个寄存器的值并记录：

8.1 按下 CLR 复位按钮；

8.2 将操作模式开关 SWC ~ SWA 设置为 011，表示“读寄存器”模式；

8.3 按下 2 次 QD，分别读出四个寄存器的值，见表 4-3 单拍跟踪结果记录表；

9. 读取存储器 12H 和 13H 的值并记录：

9.1 按下 CLR 复位按钮；

9.2 将操作模式开关 SWC ~ SWA 设置为 010，表示“读存储器”模式；

9.3 按下 QD，将数据开关设置为 12H，按下 QD，将 12H 写入地址寄存器 AR，读出 12H 的内容；再次按下 QD，读出 13H 的内容；

9.4 存储器内容同样见 表 4-3 实验结果记录表；

【连续模式】

10. 将存储器 12H 单元的内容和寄存器 R2 和 R3 中的内容复原，具体操作同上；

11. 将 DP 设置为 0，进入连续模式，按一次 QD，操作与单拍模式类似，结果见 表 4-3 实验结果记录表；

11. 断电，整理实验仪器，实验结束；

指令	μA	N μA	P	INS	PC	AR	IR	A	B	D	解释
	000001	100000	00000	01010011	00H	00H	00000000				
LD R0,[R3]	100101	001110	00000	01001100	01H	00H	01010011		0FH	0FH	从R3存的0FH地址单元读出数85H存入R0
	001110	000001	10000	01001100		0FH	01010011		0FH	85H	
	000001	100000	00000	01001100		0FH	01010011	85H	0FH		
INC R3	100100	000001	10000	01010111	02H	0FH	01001100	0FH	85H	10H	R3=0FH+1=10H
	000001	100000	10000	01010111		0FH	01001100	10H	85H		
LD R1,[R3]	100101	001110	00000	00100001	03H	0FH	01010111		10H	10H	从R3存的10H地址单元读出数23H存入R1
	001110	000001	10000	00100001		10H	01010111		10H	23H	
	000001	100000	00000	00100001		10H	01010111	23H	10H		
SUB R0,R1	100010	000001	10000	10000110	04H	10H	00100001	85H	23H	62H	执行减法85H-23H=62H存入R0
	000001	100000	00000	10000110		10H	00100001	62H	23H		
JZ 0BH	101000	010010	00000	01101000	05H	10H	10000110	23H	12H		减法结果不为0，并不产生跳转
	010010	000001	10000	01101000		10H	10000110	23H	12H		
	000001	100000	00000	01101000		10H	10000110	23H	12H		
ST R0,[R2]	100110	010000	00000	01001100	06H	10H	01101000	12H	62H	12H	将R0存的62H存入R2存的12H地址单元
	010000	000001	10000	01001100		12H	01101000	12H	62H	62H	
	000001	100000	00000	01001100		12H	01101000	12H	62H		
INC R3	100100	000001	10000	01010011	07H	12H	01001100	10H	62H	11H	R3=10H+1=11H
	000001	100000	10000	01010011		12H	01001100	11H	62H		
LD R0,[R3]	100101	001110	00000	00010001	08H	12H	01010011	62H	11H	11H	从R3存的11H地址单元读出数EFH存入R0
	001110	000001	10000	00010001		11H	01010011	62H	11H	EFH	
	000001	100000	00000	00010001		11H	01010011	EFH	11H		
ADD R0,R1	100001	000001	10000	01110010	09H	11H	00010001	EFH	23H	12H	执行加法EFH+23H=12H存入R0
	000001	100000	00000	01110010		11H	00010001	12H	23H		
JC 0CH	100111	010010	00000	01001000	0AH	11H	01110010	12H	12H		产生进位，跳转至0CH地址处的指令继续执行
	010010	000001	10000	01001000		11H	01110010	12H	12H		
	000001	100000	00000	00110001		11H	01110010	12H	12H		
AND R0,R1	100011	000001	10000	10100010	0DH	11H	00110001	12H	23H	02H	执行按位与12H&23H=02H存入R0
	000001	100000	00000	10100010		11H	00110001	02H	23H		
OUT R2	101010	000001	10000	11100000	0EH	11H	10100010	02H	12H	12H	输出R2存的12H到数据总线上
	000001	100000	00000	11100000		11H	10100010	02H	12H		
STP	101110	000001	10000	10000101	0FH	11H	11100000	02H	12H		停机
	000001	100000	00000	10000101		11H	11100000	02H	12H		

表 4-2 单拍跟踪过程记录表

说明：实验过程中所用实验箱的 P1 指示灯存在不亮的问题。

实验结果与分析讨论：

寄存器/地址		R0	R1	R2	R3	12H	13H
内容/值	程序执行前	00H	00H	12H	0FH	00H	00H
	程序执行后	02H	23H	12H	11H	62H	00H

表 4-3 实验结果记录表

连续模式下，进行复位操作后，按一次 QD，此时观察到 PC 直接变为 0FH，即程序在执行 STP 停机指令后，正确停下。

说明：连续操作和单拍操作的结果是一致的，再次验证了程序执行的正确性。

关于给定程序的功能、指令执行的分析见 表 4-2 中解释一栏中的内容；理论分析的程序执行过程与实际跟踪的运行过程一致。

微地址和后继微地址的跟踪结果均与实验四中的图 3-1 微程序流程图 一致。

关于指令寄存器 IR 和 指令通路 INS 的讨论：

指令寄存器 IR 中存放的是当前正在执行的指令内容。当前指令在执行时，程序计数器 PC 已经计算出下一条指令的地址，同时从存储器读出发送到指令通路上。所以指令通路上保存的是下一条指令的内容。（当发送条件转移时，会在当前指令执行时重新计算跳转的地址）。

从实验的跟踪过程也可以验证，指令通路 INS 上的内容比指令寄存器 IR 中的内容领先一条指令。

关于条件跳转指令的说明：

从实验过程记录表中加粗的内容可见，跳转指令执行时，会通过 PCADD 更新 PC，从而实现跳转。

四、 实验收获及体会

本次实验中，一开始在单步操作下跟踪程序执行的过程中，在执行到 INC 指令时，发现微地址与预期不符，检查此时的有关信号灯之后，对照微程序流程图，发现此时实际执行的是 LD 指令，实验箱存在被错误编程的问题。

更换实验箱后，顺利解决上述问题并完成实验。通过实验五，加深了我对 CPU 的组成结构的理解，进一步熟悉了对于存储器和寄存器的读写操作，尤其是掌握了计算机如何通过取指令、执行指令、以及执行完当前指令取下一条指令来执行程序的过程。同时通过单步跟踪理解了一条机器指令对应的若干条微指令序列组成的微程序的实现过程。对于计算机组成原理理论课的学习非常有帮助。

实验六、中断原理实验

一、 实验任务及目的

(1) 实验任务：

- 理解中断相关指令，以及每个信号的意义和变化条件；
- 将主程序和中断服务程序手工汇编成二进制机器代码；
- 通过简单的连线构成能够运行程序的 TEC-8 模型计算机；
- 将主程序和中断服务程序装入存储器，给寄存器 R1 赋初值 01H，R0 赋初值 0；
- 执行三遍主程序和中断服务程序，详细记录中断有关信号变化情况，特别记录好断点和 R0 的值；
- 将主程序中地址为 00H 的 EI 指令改为 DI，重新运行程序，记录现象；

(2) 实验目的：

- 从硬件、软件结合的角度，模拟中断的过程；
- 通过简单的中断程序，掌握中断控制器、中断向量和中断屏蔽等相关概念；
- 了解微程序控制器与中断控制器协调的基本原理；
- 掌握中断子程序和一般子程序的本质区别,掌握中断的突发性和随机性；

二、 程序的手动汇编结果（包括主程序和中断服务程序）

通过参考 Tec-Plus/Tec-8 模型计算机的指令系统，对给定的程序进行手动汇编，主程序和中断程序的手动汇编结果如表 2-1 所示：

主程序机器代码			中断程序机器代码		
地址	指令	二进制机器代码	地址	指令	二进制机器代码
00H	EI/DI	1101 0000	45H	ADD R0,R0	0001 0000
01H	INC R0	0100 0000	46H	EI	1101 0000
02H	INC R0	0100 0000	47H	IRET	1011 0000
03H	INC R0	0100 0000			
04H	INC R0	0100 0000			
05H	INC R0	0100 0000			
06H	INC R0	0100 0000			
07H	INC R0	0100 0000			
08H	INC R0	0100 0000			
09H	JMP [R1]	1001 0100			

表 2-1 程序的手动汇编结果

三、实验原理分析

(1) TEC-8 模型计算机的中断机制：

在 TEC-8 模型计算机中，中断机制用于处理外部或内部设备对 CPU 的请求，使 CPU 能够暂时中断当前程序的执行，转而执行中断服务程序，处理完毕后再返回原程序继续执行。

TEC-8 模型计算机的指令系统中，DI 关中断指令用于禁止中断，DI 执行后，CPU 不再响应新的中断请求；EI 开中断指令用于允许中断，EI 执行后，CPU 响应中断请求。

在时序发生器中，设置了一个允许中断触发器 EN_INT，其 VHDL 描述如下：

```
1. INT_EN_P: process(CLR#,MF,INTEN,INTDI,PULSE,EN_INT)
2.   begin
3.     if CLR# = '0' then
4.       EN_INT<= '0';
5.     else if MF'event and MF = '1' then
6.       EN_INT <= INTEN or (EN_INT and (not INTDI));
7.     end if;
8.     INT <= EN_INT and PULSE;
9.   end process;
```

- CLR 复位信号会将 EN_INT 复位为 0；
- 当 EN_INT=1 时，允许中断；此时按下 PULSE 产生中断请求，使 INT=1，即时序发生电路向微程序控制器输出的中断程序执行信号；
- 当 EN_INT=0 时，禁止中断；

阅读代码 `EN\_INT <= INTEN or (EN\_INT and (not INTDI))` 可知，当 EI 指令有效 (INTEN=1) 时即使能中断；否则，只有当当前中断是开启的且没有被 DI 屏蔽，才保持中断开启。

中断程序执行完毕后，需要恢复中断前的 PC 值，即中断断点的地址，使之能返回主程序继续执行。为此，设置了一个中断地址寄存器 IAR：

1. 当 LIAR = 1 时，在 T3 的上升沿，将 PC 保存在 IAR 中。
2. 当 IABUS = 1 时，IABUS 中保存的 PC 送数据总线 DBUS；

TEC-8 模型只有一个中断地址寄存器 IAR，所以只支持一级中断；

执行中断程序还需要中断服务程序的入口地址，即中断向量，本实验中，通过数据开关 SD7 ~ SD0 提供；

(2) 中断的处理流程：

关于微程序的流程图，见实验四，此处不再展示。

由流程图可见, TEC-8 模型计算机的所有指令(除 EI 和 DI 之外)在执行完毕后均进入 P4 判断位。如果 $INT=0$, 即没有中断请求, 则继续进行当前程序的正常取指阶段; 如果 $INT=1$, 即当前需要响应中断请求, 则转入微地址 11 处, 进行中断处理。

微地址 11 处的微操作, 即控制信号包括: INTDI, 用于禁止响应新的中断; LIAR, 用于将当前程序计数器 PC 的值存入中断地址寄存器; STOP, 用于等待用户通过数据开关输入中断向量。

当数据开关设置中断向量后进入微地址 12, 控制信号包括 SBUS 和 LPC, 将中断服务程序的入口地址写入程序计数器 PC, 开始中断服务程序的执行过程。

当中断服务程序执行到中断返回指令 IRET, 这条指令的功能是返回断点。进入微地址 2B 处, 微操作包括 LABUS 和 LPC, 将中断地址寄存器中保存的中断断点写入程序计数器 PC, 即“跳转”回主程序的断点处, 恢复被中断的程序。

(3) 实验过程中临时想到的问题:

实验过程中, 想到 TEC-8 模型计算机的中断信号 PULSE 按下后, 会立刻使得 $INT=1$, 即中断触发器是一个高电平而非时钟脉冲。那么假如我在 INC 指令取完指并更新了 PC, 但还没有执行 INC 的递增微操作时, 此时按下 PULSE 会不会产生这条 INC 指令实际并未执行, 中断向量保存的是下一条指令的地址的问题?

回顾微程序流程后, 发现担心是多余的。INT 信号是在取指阶段进行判定的, 所以每条指令要么执行完了才进入 P4, 即中断请求的判定, 要么在取指, 即更新 PC 前, 进行判定。所以在指令的执行过程中产生中断请求, 这条指令也能被正确执行完成。

四、 实验过程及结果

实验过程:

1. 实验准备: 断电后, 切换控制器转换开关至“微程序”, 编程开关至“0”, $DP=1$ (先进入单拍模式, 将主程序和中断服务程序写入存储器), 电路连线如表 4-1 所示:

数据通路	IR4-I	IR5-I	IR6-I	IR7-I	C-I	Z-I	CS0
开关	IR4-O	IR5-O	IR6-O	IR7-O	C-O	Z-O	GND
接线颜色	蓝	蓝	蓝	蓝	蓝	蓝	红

表 4-1 实际接线情况

- 2. 打开电源, 进入单拍模式;
- 3. 将主程序写入存储器:

3.1 按下 CLR 复位按钮;

3.2 将操作模式开关 SWC ~ SWA 设置为 001, 表示“写存储器”模式;

3.3 按下 QD, 将数据开关设置为 00H, 再一次按下 QD, 将 00H 写入地址寄存器 AR;

3.4 按照表 2-1, 依次将数据开关设置为每条指令的二进制码, 并按下 QD 写入存储器的对应地址单元;

4. 将中断服务程序写入存储器:

4.1 按下 CLR 复位按钮;

4.2 将操作模式开关 SWC ~ SWA 设置为 001, 表示“写存储器”模式;

4.3 按下 QD, 将数据开关设置为 45H, 再一次按下 QD, 将 45H 写入地址寄存器 AR;

4.4 按照表 2-1, 依次将数据开关设置为每条指令的二进制码, 并按下 QD 写入存储器的对应地址单元;

5. 通过写寄存器操作, 将数 00H 写入寄存器 R0, 01H 写入寄存器 R1:

5.1 按下 CLR 复位按钮;

5.2 将操作模式开关 SWC ~ SWA 设置为 100, 表示“写寄存器”模式;

5.3 按下 QD, 将数据开关设置 00H, 按下 QD, 写入 R0; 再将数据开关设置为 01H, 按下 QD, 写入 R1; 将开数据开关设置为任意, 重复按下 2 次 QD;

6. 执行主程序和中断服务程序:

6.1 设置 DP=0, 进入连续模式, 按下 CLR 复位按钮;

6.2 按下 QD, 主程序开始执行 (主程序为一个无限循环的程序);

6.3 按下 PULSE, 产生中断请求, 中断主程序的运行, 记录中断断点 (通过 PC7 ~ PC0 读出) 和 R0 寄存器的值 (通过 A7 ~ A0) 读出, 结果见 表 4-2 实验结果记录表;

6.4 设置 DP = 1, 进入单拍模式, 将数据开关设置为 45H, 按下 QD, 跟踪中断服务程序的执行;

6.5 按照步骤 6.1 ~ 6.4 重复进行 2 遍;

7. 探索禁止中断的情况:

7.1 将存储器 00H 的指令改为 DI, 按照步骤 6, 重做一遍, 记录发生的现象, 结果见 实验结果及分析讨论部分。

实验结果及分析讨论:

实验过程中, 发现程序的执行速度非常快, 第一遍在按下 QD 启动程序后, 立刻按下 PULSE, R0 已经递增到了 24H。具体的结果如表 4-2 实验结果记录表所示:

执行程序顺序	PC断点值	中断时的R0	跟踪过程记录	PC7 ~ PC0	A7 ~ A0
第1遍	05H	24H	第1遍	05H→45H→46H→47H→48H→05H	24H→48H
第2遍	04H	5BH	第2遍	04H→45H→46H→47H→48H→04H	5BH→B6H
第3遍	02H	54H	第3遍	02H→45H→46H→47H→48H→02H	54H→A8H

表 4-3 实验结果记录表

注：

1. 跟踪过程中，48H 实际是 IRET 指令执行过程中的一个中间状态，在指令执行过程中被更新为 05H，实现跳转回原程序；
2. A7 ~ A0 只记录了进入中断程序前和执行完中断程序后的值，省略了中间状态；
3. 上述重复执行的 3 遍过程，只在每一遍开始进行 CLR 复位操作，使程序从头开始执行，没有重新将 R0 恢复为 00H；

关于实验结果正确性的理论分析：

第一遍，程序执行一次循环， $R0 + 8$ ，PC 断点为 05H，说明此时 R0 的结果应该是 8 的倍数加 4， $24H = 32 + 4 = 4 * 8 + 4$ ，与实验结果一致；

在执行中断程序后， $R0 = 2R0 = 48H$ ，与实验结果一致；

第二遍，程序从头开始执行（注意不是从 05H 继续执行），所以第二遍中断时的 R0-48H 应该也符合上述条件，即差值为 8 的倍数 + 3， $5BH - 48H = 13H = 19 = 2 * 8 + 3$ ，与实验结果一致；

在执行中断程序后， $R0 = 2R0 = B6H$ ，与实验结果一致；

（第三遍，按下 PULSE 的间隔较长）

第三遍，程序从头开始执行（注意不是从 04H 继续执行），所以第三遍中断时的 R0-B6H 应该也符合上述条件，即差值为 8 的倍数 + 1。但是注意此时，由于 TEC-8 的寄存器只有 8 位，递增超过 FFH 会发生截断，所以正确的过程应该是 $54H + 100H - 5BH = F9H = 31 * 8 + 1$ ，与实验结果一致；

实验结果与理论分析完全一致。

屏蔽中断的实验结果：

将存储器 00H 的指令改为 DI，主程序屏蔽中断，所以按下 PULSE，并不会跳转到中断服务程序，而是正常执行主程序。

关于设置多级断点的尝试：

在中断服务程序执行的过程中，我尝试设置在中断服务程序内部设置新的断点，但发现这样会导致程序，执行时会在 47H 和 48H 两个地址间跳跃而不回到主程序断点处。

这印证了实验原理中的分析，在 TEC - 8 模型计算机中并不支持多级中断，由于只有一个 IAR 中断寄存器，如果在当前中断服务程序中设置新的断点，那么当前 PC 值会被存入 IAR 中断寄存器，覆盖主程序的中断断点，导致程序无法回到主程序继续执行。

五、 实验收获及体会

本次实验中，通过对中断服务程序的追踪，加深了我对中断请求的产生、处理的整个过程的理解，进一步理解了中断控制的机制。

通过本次实验，我掌握了中断、中断向量、中断屏蔽等概念，熟悉了中断控制器和微程序控制器在中断过程中的作用，加深了对中断的理解。同时，还巩固了对于微程序执行的跟踪，熟悉了中断程序和一般程序的执行过程。

通过前期的理论分析，在实验过程中很好地检验了我的一些思考，从而加深了我对中断原理的理解，也让我体会到了理论结合实践、实践是检验真理的唯一标准的必要性。

拓展实验

一、改造思路

(1) 实验五程序的大致功能分析:

1. 从地址 R3 开始读取两个数据:
 - $R0 = \text{mem}[R3];$
 - $R1 = \text{mem}[++R3];$
2. 做差 $R0 - R1$, 若相等则跳转 (不再存结果, 直接跳下一步);
3. 若不相等, 将差值 R0 存到 $\text{mem}[R2]$, 并递增 R3 再读取一项 $R0 = \text{mem}[R3+1];$
4. 然后 $R0 = R0 + R1$, 如果产生进位, 跳到 AND 操作;
5. 否则 $R2++$, 并 $\text{mem}[R2] = R2$;
6. 最后: $R0 = R0 \& R1$, 输出 R2;

(2) 改造面临的问题:

实验要求通过 JMP 指令, 使主程序处于循环状态, 显然需要有一个固定的寄存器存放程序的开始地址或者存放指向程序的开始地址的地址。

该寄存器固定且值在执行的过程中不能发生改变, 或者通过别的指令操作, 恢复寄存器的值, 但后者显然是复杂的, 并且由于 TEC-8 指令系统只支持从寄存器中取数或从寄存器指定的地址单元取数, 不支持立即数, 这大大加大了程序的难度。

由于 TEC-8 模型机的通用寄存器仅有 4 个, 且考虑到程序本身的功能不能改变。程序执行过程中, 虽然 R0 和 R1 只用来存放中间结果, 但由于 TEC-8 简单的指令系统导致恢复寄存器原来的数的操作是十分困难的。

再看 R2 和 R3, 两者事实上是指定了一段连续的内存单元, 在程序执行过程中用于递增取操作数。

所以, 考虑到将 R2 和 R3 实现的功能压缩为一个寄存器来实现, 另一个寄存器存储 01H (现在的 00H 被用来存放 EI 指令)。

由于原程序在一段连续的内存单元上取操作数, 联想到实际计算机内存中“堆栈”的结构, 将一个寄存器作为“栈顶”, 通过递增、递减实现原先的取数操作和恢复操作;

TEC-8 的指令系统并没有递减的指令, 但由于另一个寄存器中刚好存储了 01H, 所以可以通过两个寄存器作减法运算实现递减的功能。

至此, 算是在完美保留了原程序的执行顺序和功能的情况下, 实现了循环的操作。

二、设计的程序及程序的手动汇编结果

主程序机器代码				
地址	指令	对应原程序	二进制机器代码	备注
00H	EI		1101 0000	允许中断
01H	LD R0,[R2]	LD R0,[R3]	0101 0010	当前R2存放数据区域的开始地址 (原先是R3)
02H	INC R2	INC R3	0100 1000	此时R2+1, 需要注意在后面进行恢复 (原先是R3)
03H	LD R1,[R2]	LD R1,[R3]	0101 0110	取数
04H	SUB R2,R3	恢复用	0010 1011	由于后面要使用到原先的R2, 为避免 出错, 先恢复, 保持R2始终指向 数据区域的开始地址
05H	SUB R0,R1	SUB R0,R1	0010 0001	
06H	JZ 15H	JZ 0BH	1000 1110	跳转的地址因为程序整体下移需要 改变 offset=15H-07H=0EH
07H	INC R2	原程序中R2初始= 数据区域开始地址 +3	0100 1000	原先的功能是mem[R2]=R0, 注意 这应该是原来的R2, 现在应该是 R2+12H-0FH=R2+3
08H	INC R2		0100 1000	
09H	INC R2		0100 1000	
0AH	ST R0,[R2]		0110 1000	
0BH	SUB R2,R3	INC R3	0010 1011	注意这里原先用的是R3寄存器, R3 在开头递增过一次, 但是现在使用 单个R2寄存器固定指向数据的开始 区域, 在递增操作后被恢复, 所以 正确操作是数据区域的开始地址 +2, 即R2递增两次, 结合刚刚的操 作, 所以-1即可
0CH	LD R0,[R2]	LD R0,[R3]	0101 0010	
0DH	SUB R2,R3	恢复用	0010 1011	为避免出错, 直接恢复, 保持R2始 终指向数据区域的开始地址
0EH	SUB R2,R3		0010 1011	
0FH	ADD R0,R1	ADD R0,R1	0001 0001	
10H	JC 16H	JC 0CH	0111 0101	跳转的地址因为程序整体下移需要 改变 offset=16H-11H=05H
11H	INC R2	INC R2	0100 1000	原先的功能是R2++, mem[R2]=R2, 注意这应该是原来 的R2, 现在应该是R2+12H-0FH + 1=R2+4
12H	INC R2		0100 1000	
13H	INC R2		0100 1000	
14H	INC R2		0100 1000	
15H	ST R2,[R2]	ST R2,[R2]	0110 1010	
16H	AND R0,R1	AND R0,R1	0011 0001	
17H	OUT R2	OUT R2	1010 0010	原来的功能是输出R2, 现在的正确 地址已经在刚刚更新
18H	SUB R2,R3	恢复用	0010 1011	恢复R2指向数据区域的开始位置
19H	SUB R2,R3		0010 1011	
1AH	SUB R2,R3		0010 1011	
1BH	SUB R2,R3		0010 1011	
1CH	JMP R3	STP	1001 1100	跳转回开头, 实现循环
1DH	85H	数据区域	1000 0101	
1EH	23H		0010 0011	
1FH	EFH		1110 1111	
20H	00H		0000 0000	
21H	00H		0000 0000	

表 2-1 设计的程序及汇编结果

中断服务程序与实验六一致，此处不再赘述。中断服务程序中只会改变 R0，所以不会对指针的恢复产生影响。

我设计的程序的关键点是：

- (1) 将原先 R2 和 R3 寄存器的功能合并为 R2，用 R2 始终保存数据区域的开始位置，在程序运行时通过 R2 的递增递减实现访存，访存完毕后立刻恢复；
 - (2) 原先 R2 的功能通过 $R2 + 3$ 偏移量进行代替；
 - (3) 当前 R3 寄存器用于保存程序的循环入口地址，即 01H，顺便实现递减的操作；
- 设计的程序完美保留了原程序的执行顺序和逻辑，并实现了循环。

改造后初始寄存器中存放的数据是：

寄存器/地址		R0	R1	R2	R3
内容/值	程序执行前	00H	00H	1DH	01H

表 2-2 初始寄存器存放的数据

三、 实验过程及结果

实验过程：

1. 实验准备：断电后，切换控制器转换开关至“微程序”，编程开关至“0”，DP = 1（先进入单拍模式，将主程序和中断服务程序写入存储器），电路连线如表 4-1 所示：

数据通路	IR4-I	IR5-I	IR6-I	IR7-I	C-I	Z-I	CS0
开关	IR4-O	IR5-O	IR6-O	IR7-O	C-O	Z-O	GND
接线颜色	蓝	蓝	蓝	蓝	蓝	蓝	红

表 4-1 实际接线情况

- 2. 打开电源，进入单拍模式；
- 3. 将主程序写入存储器：
 - 3.1 按下 CLR 复位按钮；
 - 3.2 将操作模式开关 SWC ~ SWA 设置为 001，表示“写存储器”模式；
 - 3.3 按下 QD，将数据开关设置为 00H，再一次按下 QD，将 00H 写入地址寄存器 AR；
 - 3.4 按照表 2-1，依次将数据开关设置为每条指令的二进制码，并按下 QD 写入存储器的对应地址单元；
- 4. 将中断服务程序写入存储器：
 - 4.1 按下 CLR 复位按钮；
 - 4.2 将操作模式开关 SWC ~ SWA 设置为 001，表示“写存储器”模式；

4.3 按下 QD，将数据开关设置为 45H，再一次按下 QD，将 45H 写入地址寄存器 AR；

4.4 按照表 2-1，依次将数据开关设置为每条指令的二进制码，并按下 QD 写入存储器的对应地址单元；

5. 通过写寄存器操作，将数 1DH 写入寄存器 R2，01H 写入寄存器 R3：

5.1 按下 CLR 复位按钮；

5.2 将操作模式开关 SWC ~ SWA 设置为 100，表示“写寄存器”模式；

5.3 按下 QD，将数据开关设置为任意值，按下 QD，重复 2 次（写寄存器是按顺序依次写入，所以跳过 R0、R1）；

5.4 将数据开关设置分别设置为 1DH 和 01H 并按下 QD，将数据写入 R2 和 R3；

6. 执行主程序和中断服务程序：

6.1 设置 DP=0，进入连续模式，按下 CLR 复位按钮；

6.2 按下 QD，主程序开始执行（主程序为一个无限循环的程序）；

6.3 按下 PULSE，产生中断请求，中断主程序的运行，记录相关结果，结果见 表 4-3 实验结果记录表；

6.4 设置 DP = 1，进入单拍模式，将数据开关设置为 45H，按下 QD，跟踪中断服务程序的执行；

6.5 继续通过单拍模式完成主程序的本次循环；

实验结果及分析讨论：

由于时间有限，该拓展实验仅进行了一次中断操作，再跟踪完成中断操作后，继续单步执行完主程序的一次循环后，读出有关寄存器和存储器的内容，并记录了结果，见 表 4-2 和 表 4-3。

执行程序顺序	PC断点值	跟踪过程记录	PC7 ~ PC0
第1遍	05H	第1遍	05H→45H→46H→47H→48H→05H

表 4-2 跟踪过程记录

寄存器/地址		R0	R1	R2	R3	1DH	1EH	1FH	20H	21H
内容/值	中断一次后再完成一次循环	02H	23H	1DH	01H	85H	23H	EFH	C4H	00H

表 4-3 循环中途中断一次后继续完成循环后的有关结果

实验结果的理论分析：

事实上，简单分析程序即可发现，由于我们遵循了实验五的程序，固定数据区域，R0 ~ R1 作

为中间结果的寄存器，每次循环都被重新写入，而数据区域开始地址处的 3 个内存单元的数据并不会被重写，所以每次取的运算操作数都一致，所以理论上，每次循环执行后，寄存器和存储器的内容始终为：

寄存器/地址		R0	R1	R2	R3	1DH	1EH	1FH	20H	21H
内容/值	一次循环执行后	02H	23H	1DH	01H	85H	23H	EFH	62H	00H

当中断服务程序发生时，由于中断服务程序会改变 R0 的结果，可能导致循环执行的流程出现区别。

实验过程中，在 05H 处中断，进入中断程序， $R0 = 2R0 = 2 * 62H = C4H$ ，返回主程序后，通过 'STR0,[R2]' 指令被写入 20H 地址单元，之后由于 R0 又被存入 EFH，后面的结果与实验五一致。中断操作仅改变了 20H 地址单元上的数为 C4H，与实验结果一致。

四、实验收获及体会

本次实验中，由于 TEC-8 模型计算机的有限的资源（包括通用寄存器数量、寄存器位数、指令系统等），在设计扩展程序上花费了较长的时间。本来的想法是只保留原程序的一部分功能，想办法删除部分设计改动寄存器的指令。通过对程序执行过程中每个寄存器作用的仔细分析，最后想出用一个寄存器存放指向数据区域开始地址，将 2 个寄存器的功能压缩为 1 个，才完美保留了原程序的执行顺序和逻辑。

通过自主设计程序，也让我再次体会到计算机系统设计的精妙之处，程序在翻译成机器指令的过程中，编译器在协调可供分配的寄存器和其他资源时，需要进行优化，这其中需要复杂的设计。

同时，通过拓展实验，进一步掌握了中断原理。有效完成了实验任务并达到了实验要求。